

# 조세훈

**Frontend Engineer** | React · Next.js · React Native

Tel: 010-8968-9274

Email: tpgnsjth@gmail.com

GitHub: [github.com/951jth/pandytalk](https://github.com/951jth/pandytalk) | Google Play: [PandyTalk](#)

---

## 경력

(주)델피콤 | Frontend Engineer | FE 3명, BE 3명

2021.05 ~ 현재 (6년차)

- B2B 050 가상번호 통신 플랫폼(BizCall) 관리자 웹 및 랜딩 페이지 개발·운영
  - 모즐(Mozzle) 모임 관리 앱 및 관리자 웹 기능 개발·운영
  - 신규 기능 개발 및 프로젝트 단위 시스템 구축
- 

## 프로필

React·Next.js 기반 관리자 웹에서 대량 데이터를 다루며 발생하는 렌더링·인터랙션·데이터 처리 병목을 해결해 온 6년 차 프론트엔드 개발자입니다. Virtualization·Memoization·Web Worker 등의 기술을 적용하고 전후 성능을 측정하며, 데이터 규모가 증가해도 안정적으로 사용할 수 있는 프론트엔드 구조를 만들어 왔습니다.

---

## 보유 기술

- **Performance:** Virtualization, Memoization, Web Worker, Chrome Performance, Lighthouse
- **Data & State:** Zustand, React Query, MobX, SQLite, Firebase (Firestore, FCM)
- **Frontend:** TypeScript, React, Next.js, React Native (CLI)
- **UI & Styling:** Tailwind CSS, MUI, Ant Design
- **Tools & 협업:** Figma, Git, Slack

---

# 회사 프로젝트 경험

---

## 1. Mozzle – 앱 내 WebView 웹페이지·관리자 페이지 구축 및 운영

모임 관리 서비스 (2023.02 ~ 현재)

Mozzle은 회원명부, 운영진, 일정, 회비·결산, 게시판 등을 관리할 수 있는 단체·모임 운영 서비스입니다.

앱 내 WebView 웹페이지와 관리자 웹의 기능 개발·운영을 담당했으며, 대표적으로 회원명부 편집 과정에서 발생한 입력 반응 지연과 렌더링 병목을 분석하고, 1차 개선의 한계를 바탕으로 렌더링 및 편집 상태 구조를 단계적으로 재설계했습니다.

사용 기술: [React](#), [MUI](#), [React Virtuoso](#), [Immer](#)

### 모임 운영진 페이지의 회원명부 편집 구조 개선

#### 문제 상황

- 1,000명·22개 이상 컬럼의 회원명부에서 입력 시 전체 목록 상태가 갱신되며 **전체 Row·Cell의 렌더링 로직이 다시 실행**, 검색어 3글자 입력의 UI 반응 시간이 약 **1,330ms**까지 지연되었습니다.
- 모든 Row를 한 번에 DOM에 유지하면서 약 **54,458개의 DOM Node**가 생성되어 초기 렌더링과 3,000건 스크롤에서 Long Task가 발생했습니다.
- 1차로 Uncontrolled Input과 500건 단위 점진 렌더링을 적용해 입력 반응을 개선했지만, 편집값이 DOM과 데이터로 분리되고 스크롤할수록 Row가 누적되어 DOM Node가 최대 **220,506개**까지 증가하는 한계가 있었습니다.

#### 해결

- 입력값을 Cell 내부 상태에서 처리하고 Blur·Enter 시점에만 상위 데이터로 반영해 **키 입력이 전체 테이블 상태 변경으로 확산되지 않도록** 구조를 변경했습니다.
- Immer의 Structural Sharing과 Row 단위 Memoization을 적용해 **변경된 Row만 새 참조로 갱신**하고 나머지 Row는 렌더링을 건너뛰도록 개선했습니다.
- 편집값과 원본 데이터를 회원 ID 기준으로 관리하고, React Virtuoso를 적용해 **전체 데이터는 유지**하되 **화면 주변 Row만 DOM에 렌더링**하도록 변경했습니다.

#### 성과

- 서버 응답 이후 1,000건 데이터를 화면에 표시하는 클라이언트 렌더링 시간을 **4,918ms → 139ms**, 검색 입력 UI 반응 시간을 **1,330ms → 29ms**로 단축했습니다.

- DOM Node를 **54,458개 → 2,306~5,103개**, 3,000건 스크롤 Long Task를 **19건 → 1건**으로 감소시켰습니다.
- 개선한 편집 구조를 운영진 페이지의 회원 관리 기능으로 확장해, **회원별 모달 진입 없이 목록에서 바로 정보를 수정할 수 있도록 관리 플로우를 단순화**했습니다.

## 측정 환경

Windows 11 / Intel Core i5 / RAM 16GB / Chrome DevTools Performance / 동일 기기·데이터 조건

기술 포트폴리오: [회원명부 편집 테이블 렌더링 성능 개선기](#)

---

## 2. 050 BizCall – 관리자 웹·랜딩 페이지 개발 및 운영

가상번호 관리 웹 서비스 (2022.10 ~ 현재)

050 가상번호를 통해 고객의 실제 전화번호 노출을 방지하고, 기업 고객에게 번호·통화내역·통신 부가서비스 관리 기능을 제공하는 B2B 안심번호 서비스입니다.

관리자 웹 리뉴얼 구축·운영과 Next.js 기반 랜딩페이지 구축을 담당했습니다.

### 사용 기술

관리자 웹 – **React, CRA, SheetJS, Web Worker**

랜딩 페이지 – **Next.js**

### 2-1. 대량 콜로그 엑셀 다운로드 구조 개선

#### 문제 상황

- 백엔드 개발자로부터 대량 데이터의 콜로그 Excel 다운로드 요청 시 서버 응답 지연 이슈가 공유되었습니다.
- 다운로드 요청이 동시에 발생할 경우 엑셀 생성 작업이 서버의 다른 API 처리에도 영향을 주는 문제가 있었습니다.
- 클라이언트 생성 방식으로 전환한 이후에도 **API 직렬 요청과 메인 스레드 파일 생성**으로 대량 데이터 처리 시간이 길고 UI 응답성 저하 위험이 남았습니다.

#### 해결

- 서버는 데이터를 **페이징 단위 JSON**으로 제공하고 클라이언트에서 **Excel**을 생성하도록 구조를 전환했습니다.
- 이후 API 요청을 **제한 병렬 처리**하고 파일 생성을 **Web Worker**로 분리했으며, 데이터를 10만 건 단위 Sheet로 나눠 ZIP으로 생성하도록 개선했습니다.

- 장시간 다운로드의 처리 상태를 확인할 수 있도록 **진행률 UI**를 제공했습니다.

## 성과

- 서버 증설 없이 클라이언트 다운로드 구조로 전환해 **개발환경에서 최대 약 79.6만 건의 콜로그 다운로드를 처리**했습니다.
- 네트워크 요청과 파일 생성 작업을 분리해 **대용량 다운로드의 처리 구조와 UI 응답성을 개선**했습니다.
- 동일 개발환경의 데이터 규모별 측정에서, 20.5만 건 기준 처리시간을 **150.8초 → 90.8초**로 단축했습니다.

## 측정 환경

Windows 11 / Intel Core i5 / RAM 16GB / Chrome DevTools Performance / 동일 기기·데이터 조건

---

## 2-2. BizCall 랜딩페이지 성능 및 SEO 개선

### 문제 상황

- JavaScript 기반 반응형 분기로 초기 렌더링 이후 요소가 재배치되며 **\*\*레이아웃 시프트(CLS)\*\***가 발생했습니다.
- 대용량 배너 이미지와 과도한 클라이언트 렌더링으로 **초기 로딩 비용**이 높았습니다.

### 해결

- JS 기반 모바일 분기를 CSS 반응형 구조로 전환하고 배너 비율을 고정해 CLS를 개선했습니다
- RSC·Client Island 분리와 WebP·Dynamic Import를 적용해 초기 로딩 비용을 줄였습니다.

### 성과

- 배너 용량을 데스크톱 **710KB→109KB**, 모바일 **365KB→55KB**로 약 85% 축소했습니다.
- Lighthouse Performance **73→95**, SEO **91→100**, CLS **0.679→0.065**로 개선했습니다.
- Search Console 한 달 비교에서 노출수 **1,290→1,970건**을 확인했습니다.

포트폴리오: [BizCall 대용량 데이터 처리·웹 성능 개선기](#)

---

## 3. Payking – 결제 링크 웹·React Native 앱·관리자 웹 개발

간편 결제 앱 및 웹의 시스템 구축 (2025.06 ~ 2025.11 / FE 3명, BE 3명 등 총 8명)

별도의 단말기 없이 스마트폰만으로 대면(NFC·카메라) 및 비대면(SMS·URL) 결제를 지원하는 소상공인용 모바일 통합 결제 솔루션입니다.

### 사용 기술

결제 링크 웹 – **Next.js**

관리자 웹 – React, Vite, Tailwind CSS

모바일 앱 – React Native CLI

## 역할

- SMS·URL로 전달된 링크에서 결제를 진행하는 **Next.js 기반 비대면 결제 페이지 단독 구현**
- BFF 응답과 PG 결제 요청·결과 페이지를 연결하는 **결제 흐름 구현**
- React Native CLI 기반 결제 앱의 **정기결제·상품관리 UI 및 공통 컴포넌트 개발**
- React 기반 관리자 웹의 입력 화면 및 공통 폼 구조 개발

## 3-1. 비대면 결제 페이지 구축 및 iOS Safari 결제 오류 대응

### 문제 상황

- 운영 중 QR로 접근한 iOS Safari 환경에서 PG 결제 팝업이 열리지 않는 문제 발생
- 동일 페이지가 카카오톡 인앱 브라우저에서는 정상 동작해 브라우저 환경 차이를 중심으로 원인 분석

### 해결

- Safari 실기기에서 재현해 API 요청·응답은 정상이나, **비동기 응답 이후 PG 팝업 생성 단계에서 실패하는 흐름 확인**
- 사용자 결제 동작 시 팝업을 선점하고, API 응답 후 해당 팝업으로 PG Form을 Submit하도록 호출 순서 변경

### 성과

- 수정 후 동일 iOS Safari 환경에서 정상 결제 확인
- 

## 3-2. 관리자 입력 플로우 및 공통 Form 구조 개선

### 문제 상황

- 디자인 확정 전 다수의 관리자 입력 화면 개발이 진행되어, 화면별 반복 구현 시 이후 디자인·Validation 변경 범위가 커질 가능성이 있었음
- 기존 **상세 → 수정 → 저장**의 3단계 입력 흐름으로 불필요한 화면 상태와 구현 범위가 증가

### 해결

- 기획·PM과 요구사항을 조율해 입력 흐름을 **3단계 → 2단계로 단순화**
- 반복되는 레이아웃·Validation 정책을 **설정 기반 공통 Form 구조로 분리**하고, 화면별 UI는 **render**를 통해 확장할 수 있도록 설계
- 팀 적용 과정에서 **Form 내부 상태의 데이터 흐름이 불명확하다는 피드백을 반영**, 부모가 상태를 소유하도록 구조를 변경해 상태 변경 흐름을 명시화

## 성과

- 공통 Form 적용으로 디자인·Validation 변경 시 개별 화면 수정 범위 축소
- 검색 Form의 Enter Submit 요구사항을 공통 영역에서 반영해 **기존 24개 사용 화면의 개별 수정 없이 대응**

기술 포트폴리오: [Payking 결제·관리자 웹 문제 해결기](#)

---

## 4. 기타 프로젝트

- **델피콤 홈페이지 및 관리자 페이지 (Next.js)** | 2024.07. ~ 2024.08

NextAuth 기반 관리자 로그인과 Next.js API Route 기반 내부 API를 구현하고, MariaDB SQL 조회 및 응답 가공 처리

- **OCC 호텔·여행사 중계 플랫폼** | 2024.02 ~ 2024.06

react-i18next 기반의 다국어 리소스를 기능 및 페이지 도메인 단위로 분리하는 네임스페이스(Namespace) 방식으로 모듈화

---

## 개인 프로젝트

---

### 1. PandyTalk – React Native 기반 AI 채팅 앱 개인 프로젝트

개인 프로젝트 / 1인 개발 (2025.06 ~ 운영 중)

React Native·Firebase 기반 Android 채팅 앱을 1인 개발·배포하고, SQLite를 활용한 메시지 캐싱·동기화와 SSE 기반 AI 응답 스트리밍을 구현했습니다.

**사용 기술:** React Native CLI, Firebase (Firestore), SQLite, React Query, SSE

#### 문제 상황

- 채팅방 진입 시 메시지를 Firestore에서 반복 조회하면서 초기 로딩이 네트워크 상태에 영향을 받고 Read가 증가했습니다.
- 완성된 AI 응답을 한 번에 표시해 사용자가 전체 생성 시간 동안 기다려야 했습니다.

#### 해결 방향

- SQLite를 기본 조회 경로로 사용하고, 로컬 데이터가 부족한 경우에만 Firestore에서 필요한 페이지를 보충했습니다.
- 장기 미접속 시 누적 메시지가 한꺼번에 구독되는 것을 방지하기 위해 실시간 구독을 최신 20건으로 제한하고, React Query로 화면 캐시와 페이지네이션을 관리했습니다.
- AI 응답은 SSE로 점진 수신하고, 원문 버퍼와 화면 출력 주기를 분리했습니다.

## 성과

- 로컬 데이터가 충분하면 Firestore를 다시 조회하지 않고 기존 대화를 표시하도록 조회 구조를 개선했습니다.
- AI 답변 생성 중에도 내용을 확인할 수 있게 하고, 최종 응답만 Firestore에 저장하도록 구성했습니다.

포트폴리오: [팬디톡 채팅 구조 및 시연 영상](#)

---